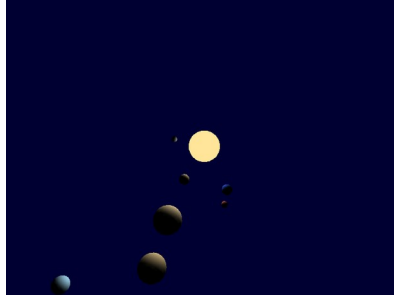


Planets (planets.*)

Escriu VS+FS per simular un mini sistema solar. No estarà a escala real perquè, altrament, seria molt difícil veure-ho tot. El VS farà tota la feina. El FS només farà les tasques per defecte.



Al fitxer **planets_defs.txt** teniu diversos array's que haureu de copiar al vostre VS:

```
const int NUM_BODIES = 9; // Sun + 8 planets

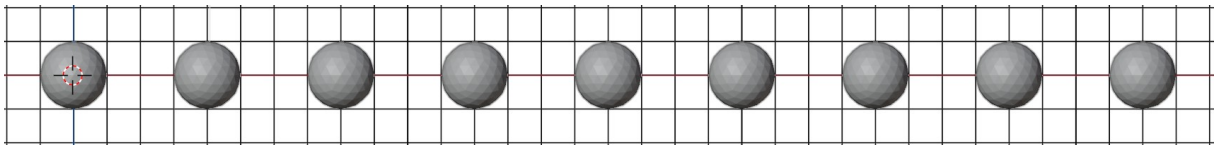
// Radi de cada planeta
const float bodyRadii[NUM_BODIES] = float[NUM_BODIES](
    2.0, // Sun
    0.3, // Mercury
    ... );

// Velocitat (rad/s) amb la que gira cada planeta al voltant del Sol
const float bodyOrbitalAngularSpeed[NUM_BODIES] = float[NUM_BODIES](
    0.0, // Sun (stationary at center)
    ... );

// Distància de cada planeta al Sol
const float bodySunDistance[NUM_BODIES] = float[NUM_BODIES](
    0.0, // Sun
    ... );

// Color RGB de cada planeta
const vec3 bodyColors[NUM_BODIES] = vec3[NUM_BODIES](
    vec3(1.0, 0.9, 0.6), // Sun
    ... );
```

Aquest problema està pensat pel model **9spheres.obj** el qual conté 9 esferes de radi 1. La primera esfera (Sol) està centrada a l'origen. La resta estan en filera, separades de l'anterior per 2 unitats en l'eix X (la distància entre el centre d'una esfera i la següent és de 4 unitats):



Cada esfera té 960 vèrtexs (`const int NUM_VERTS_SPHERE = 960;`).

El VS ha d'aplicar una transformació a cada vèrtex que dependrà de a quina de les 9 esferes pertany. Això ho podeu determinar usant `gl_VertexID`, que és un enter amb l'índex del vèrtex que està tractant el VS. Per exemple, el vèrtex pertany a l'esfera que representarà el Sol si `gl_VertexID` és un índex entre 0 i `NUM_VERTS_SPHERE - 1`. Sigui `ID` un valor enter entre 0 i 8 que indica el número d'esfera al qual pertany el vèrtex que està processant el VS. Els valors per a les transformacions de sota (radi, distància al sol...) els podeu obtenir amb `bodyRadii[ID]`, `bodySunDistance[ID]`, etc.

El VS haurà d'aplicar les següents transformacions al vèrtex en object space, per finalment passar-lo a clip space (**cadascuna la podeu implementar amb o sense matrius, com considereu més apropiat**):

1. Aplicar la translació que mou l'esfera a l'origen: serà una translació en X de $-4.0 \cdot ID$, ja que 4.0 és la separació entre centres d'esferes consecutives.

2. Escalar l'esfera d'acord amb el radi del planeta.

[Ara podríem implementar la rotació del planeta respecte el seu eix, però per a simplificar, no ho farem.]

3. Traslladar l'esfera en l'eix X, d'acord amb la seva distància al Sol.

4. Calcular l'angle de rotació (per l'òrbita al voltant del sol). L'angle en radians serà el producte de `time` per la velocitat angular del planeta (`bodyOrbitalAngularSpeed[ID]`). Com que no volem haver d'esperar anys per veure algun moviment, multiplicarem l'angle per **const float TIME_SCALE = 1e7**. Useu aquest angle per aplicar la rotació respecte l'eix Y (com que al pas 3 heu traslladat el planeta, la rotació reproduirà l'òrbita al voltant del Sol).

5. Finalment, traslladarem el vèrtex **16** unitats en l'eix X (això últim és per deixar el Sol aproximadament al centre de la caps englobant de l'escena inicial i minimitzar problemes de retallat).

En quant a il·luminació, heu de fer els següents càlculs en **world space** (que és com us haurà quedat el vèrtex després del pas 5); no useu `lightPosition`, sinó el centre del Sol:

1. Calcular el light vector unitari, tenint en compte que el Sol està centrat al (16, 0, 0) degut al pas 5.

2. Calcular el factor difós ($N \cdot L$), excepte pel Sol ($ID == 0$), que tindrà un factor de 1.0. Useu la normal en world space (li afectarà la rotació del pas 4).

3. El color final serà el resultat de multiplicar aquest factor pel color del planeta (`bodyColors[ID]`).

Com hem dit abans, el FS farà les tasques per defecte.

Identificadors obligatoris:

`planets.vert`, `planets.frag` (en minúscules)

Les constants de l'enunciat.

ditherTransparency (ditherTransparency.*)

Escriu un **vertex shader** i un **fragment shader** que simulin un efecte de **transparència falsa** fent servir tècniques de **dithering**. L'objectiu és que, en determinades zones de l'escena, alguns fragments es descartin seguint un patró, de manera que es pugui veure parcialment el que hi ha al darrere.

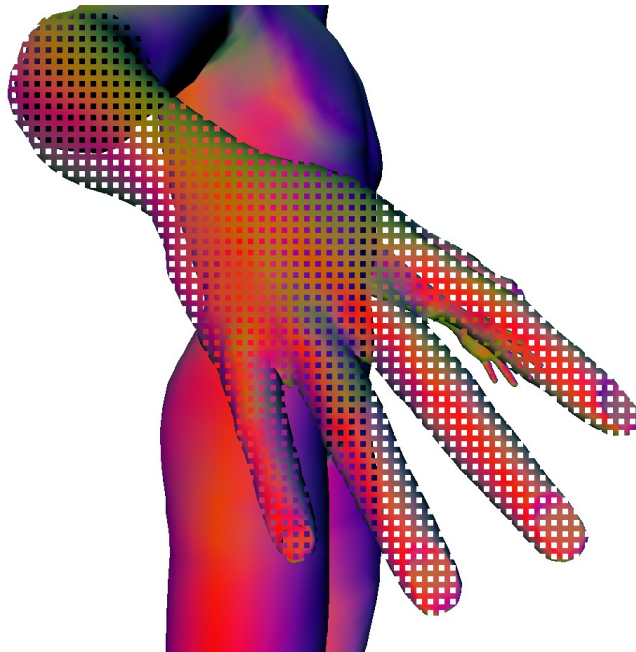
Al **vertex shader**, a més de fer les tasques per defecte, haureu de calcular la **distància del vèrtex a la càmera** i la longitud de la **diagonal de la capsa contenidora de l'escena**, ambdues en *eye space*. Aquests dos valors es passaran com a variables interpolades al fragment shader. Per la diagonal feu servir els uniforms `boundingBoxMin` i `boundingBoxMax`.

Al **fragment shader**, si la distància del fragment a la càmera és **menor o igual** a la longitud de la diagonal de la capsa contenidora multiplicada per un **ràtio «llindar de distància»** (uniform `float thresholdDistanceRatio`), llavors s'aplicarà l'efecte de transparència (*dithering*). En cas contrari, el fragment es dibuixa normalment, tal com ho farien els shaders per defecte. El paràmetre `thresholdDistanceRatio` controla fins a quina profunditat des de la càmera s'aplica l'efecte de transparència, en funció de la grandària de l'escena. [2 punts]

L'efecte tindrà dos modes controlats per un uniform `int mode`.

Per **mode==0** [4 punts] heu d'implementar un patró de *dithering* senzill basat en la posició del fragment. L'objectiu és **descartar grups de fragments en una disposició de quadrícula regular**, deixant visibles petits quadrats distribuïts de forma homogènia que permetin veure el fons.

Per aconseguir-ho, heu d'agrupar els fragments en blocs quadrats de $n \times n$, i descartar-ne un de cada quatre blocs (el que ocupa la cantonada inferior esquerra de cada grup), com es mostra a la imatge següent:



Per **mode == 1** [4 punts], la decisió de si es descarta o no un fragment a la zona afectada es farà en funció del canal **r** d'una **textura de patró** (uniform `sampler2D ditherTexture`), que podria ser:

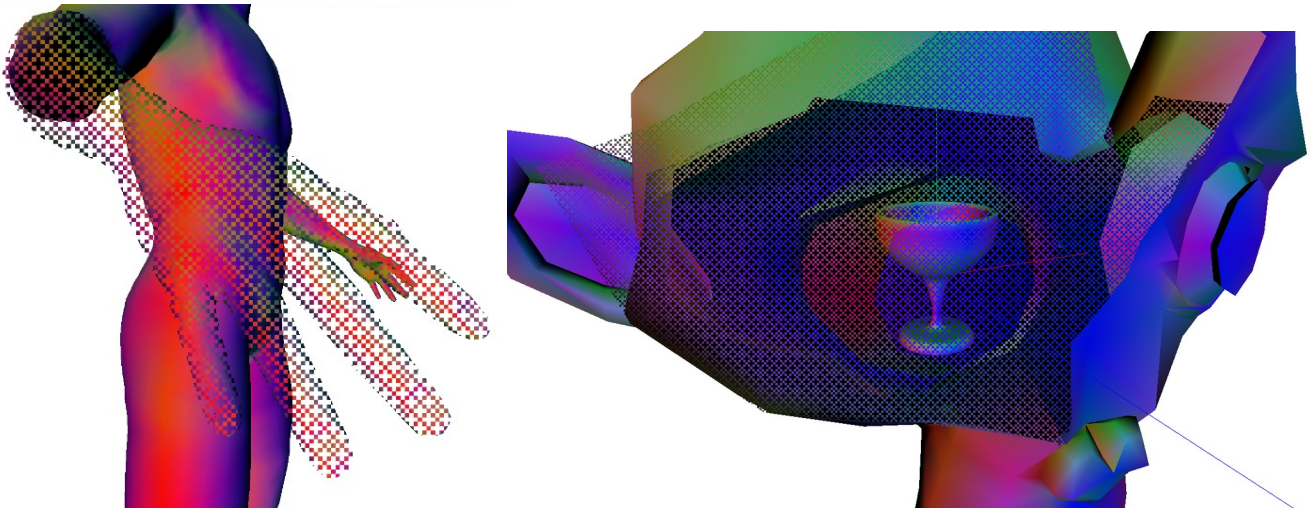
- una **Bayer matrix** (petita textura que codifica un patró ordenat de valors per simular tons grisos) com `BayerMatrix.png`, o bé
- una **textura de soroll** com `0075-1.jpg`.

Observeu que el patró codificat per la textura **no s'aplica als objectes, sinó als píxels del viewport**. Però **no volem que el patró es distorsioni** amb redimensionaments del *viewport*. Per això, hem d'accedir amb coordenades de textura calculades a partir de `gl_FragCoord.xy`, de forma que els texels no estiguin ni magnificats ni minificats. Per tant, el patró de la textura es repetirà en tot el *viewport* un nombre no necessàriament enter de vegades. Podeu fer servir que les textures que us proposem mesuren 256x256, i que cada porció del viewport de 256x256 píxels s'ha de correspondre inicialment 1-1 als texels de la textura.

En aquest cas volem, a més a més, la capacitat d'escalar el patró codificat a la textura. Per això, donat el `uniform int n`; (que ja hem fet servir més amunt) volem fer servir el patró de la textura n vegades en cada direcció, dins de cada bloc de 256x256 píxels del *viewport*. Aleshores, cada bloc de 256x256 píxels contindrà n^2 còpies del patró de *dithering*. Quan $n==1$, es tindrà la correspondència 1-1 entre *texels* i *píxels* que s'indicava més amunt.

En tots els casos, el fragment es **descartarà** si el valor del canal **r** llegit de la textura és **menor o igual** a un llindar de **0.6**.

Aquest mode hauria de produir un efecte semblant al de les imatges següents (fent servir `BayerMatrix.png`):



Nota: els colors són els mateixos que posarien els shaders per defecte.

Identificadors obligatoris:

```
ditherTransparency.vert, ditherTransparency.frag
uniform int mode = 0;
uniform int n = 4;
uniform float thresholdDistanceRatio = 0.25;
uniform vec3 boundingBoxMin;
uniform vec3 boundingBoxMax;
uniform sampler2D ditherTexture;
```

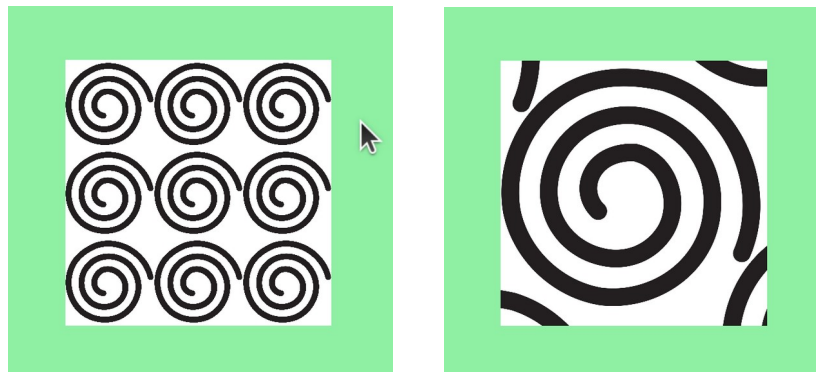
vertigen (vertigen.*)

Escriu un VS+FS que animaran la textura aplicada als objectes fent-la girar al voltant del centre de la textura. Més concretament, la textura s'aplicarà girada un angle $2\pi \sin(0.3 \cdot \text{time})$, on `time` és l'habitual `uniform float time;`. Recordeu que la matriu de rotació per un angle α és

$$\begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix}$$

i que GLSL l'emmagatzema per columnes.

També cal que, fent servir el `uniform int mult;` hom pugui modificar el nombre de còpies de la textura que es fan servir a l'interval $[0,1]$, és a dir que volem el comportament que tenim quan multipliquem les coordenades de textura per `mult`, però on cada còpia gira entorn del seu propi centre (veure el comportament esperat a `vertigen.mp4`). Aquí, a l'esquerra, veieu una captura de pantalla amb `mult==3`:



Fixeu-vos a més que el tractament comú de les textures amb *wrapping mode* `GL_REPEAT` tant en `s` com en `t` produeix resultats no volguts quan la imatge que estem rotant és aproximadament rodona. La figura de dalt, a la dreta, mostra com quan el gir és de vora 45 graus, altres porcions de la textura envaeixen l'àrea d'aquesta còpia.

Tanmateix, no podem modificar aquest comportament des dels shaders. En canvi, sí que el podem emular. Cal doncs, en tercer lloc, que quan el resultat de girar les coordenades de textura surti del rang $0..1$, el limitem a l'extrem més proper d'aquest rang (considera fer servir la funció `clamp()`). Així emularem aproximadament el comportament que resultaria d'haver triat `GL_CLAMP_TO_EDGE`.

Et suggerim que per resoldre aquest problema, procedeixis per parts:

- Implementa la rotació d'una sola còpia de la textura
- Modifica l'anterior per a incorporar `mult` còpies
- Implementa la simulació de `GL_CLAMP_TO_EDGE`

En la solució que entreguis, es valorarà la claredat, i l'aprofitament de les operacions vectorials del GLSL.

Identificadors obligatoris:

```
vertigen.vert, vertigen.frag
uniform float time = 0;
uniform sampler2D spiral;
uniform int mult = 2;
```